游戏助手服务接口说明

目录

1 游戏助手服务接口说明 **9**

2 更新日志 **9**

2.1 2026-08-13 . . . 9
2.2 2026-07-31 . . . 9
2.3 2025-09-23 . . . 9
2.4 2025-08-29 . . . 9
2.5 2025-08-11 . . . 10
2.6 2025-08-06 . . . 10
2.7 2025-08-05 . . . 10
2.8 2025-08-04 . . . 10
2.9 2025-07-25 . . . 10
2.10 2025-06-26 . . . 10
2.11 2025-06-25 . . . 10
2.12 2025-06-24 . . . 10
2.13 2025-06-06 . . . 10
2.14 2025-05-30 . . . 10
2.15 2025-05-12 . . . 10
2.16 2025-05-09 . . . 10
2.17 2025-04-29 . . . 10
2.18 2025-04-25 . . . 11
2.19 2025-04-23 . . . 11
2.20 2025-04-18 . . . 11
2.21 2025-04-16 . . . 11
2.22 2025-03-17 . . . 11
2.23 2025-03-04 . . . 11
2.24 2025-02-27 . . . 11
2.25 2025-02-18 . . . 11
2.26 2025-02-07 . . . 11
2.27 2025-01-27 . . . 11
2.28 2025-01-24 . . . 11
2.29 2025-01-23 . . . 11
2.30 2025-01-21 . . . 12
2.31 2025-01-17 . . . 12

3 关闭服务 **12**

3.1 接口 `/server/shutdown` . . . 12
3.1.1 Method `Post` . . . 12
3.1.2 Content-Type `application/json` . . . 12
3.1.3 参数 . . . 12
3.1.4 返回 . . . 12
3.1.5 例子 . . . 12
3.2 接口 `/server/shutdown_kill` . . . 12
3.2.1 Method `Post` . . . 13

3.2.2 Content-Type `application/json` . . . 13
3.2.3 参数 . . . 13
3.2.4 返回 . . . 13
3.2.5 例子 . . . 13

4 配置文件 13
4.1 接口 `/config/reload` . . . 13
4.1.1 Method `Post` . . . 13
4.1.2 Content-Type `application/json` . . . 13
4.1.3 参数 . . . 13
4.1.4 返回 . . . 13
4.1.5 例子 . . . 13

5 Vision 服务 14
5.1 接口 `/vision/service/enable` . . . 14
5.1.1 Method `Get` . . . 14
5.1.2 Content-Type `application/json` . . . 14
5.1.3 参数 . . . 14
5.1.4 返回 . . . 14
5.1.5 例子 . . . 14
5.2 接口 `/vision/service/enable/{enable}` . . . 14
5.2.1 Method `Post` . . . 14
5.2.2 Content-Type `application/json` . . . 14
5.2.3 参数 . . . 14
5.2.4 返回 . . . 14
5.2.5 例子 . . . 15
5.3 接口 `/vision/service/init/{instance_id}` . . . 15
5.3.1 Method `Post` . . . 15
5.3.2 Content-Type `application/json` . . . 15
5.3.3 参数 . . . 15
5.3.4 返回 . . . 15
5.3.5 例子 . . . 15
5.4 接口 `/vision/service/list` . . . 15
5.4.1 Method `Get` . . . 15
5.4.2 Content-Type `application/json` . . . 15
5.4.3 参数 . . . 15
5.4.4 返回 . . . 16
5.4.5 例子 . . . 16
5.5 接口 `/vision/service/delete/{instance_id}` . . . 16
5.5.1 Method `Post` . . . 16
5.5.2 Content-Type `application/json` . . . 16
5.5.3 参数 . . . 16
5.5.4 返回 . . . 16
5.5.5 例子 . . . 16
5.6 接口 `/vision/scene/insert/{instance_id}/{scene_id}` . . . 17

5.6.1 Method `Post` . . . 17
5.6.2 Content-Type `multipart/form-data` . . . 17
5.6.3 参数 . . . 17
5.6.4 返回 . . . 17
5.6.5 例子 . . . 17
5.7 接口 `/vision/scene/list/{instance_id}` . . . 18
5.7.1 Method `Get` . . . 18
5.7.2 Content-Type `application/json` . . . 18
5.7.3 参数 . . . 18
5.7.4 返回 . . . 18
5.7.5 例子 . . . 18
5.8 接口 `/vision/scene/picture/list/{instance_id}/{scene_id}` . . . 19
5.8.1 Method `GET` . . . 19
5.8.2 Content-Type `application/json` . . . 19
5.8.3 参数 . . . 19
5.8.4 返回 . . . 19
5.8.5 例子 . . . 19
5.9 接口 `/vision/scene/picture/delete/{instance_id}/{scene_id}` . . . 19
5.9.1 Method `Post` . . . 19
5.9.2 Content-Type `application/json` . . . 19
5.9.3 参数 . . . 19
5.9.4 返回 . . . 20
5.9.5 例子 . . . 20
5.10 接口 `/vision/scene/delete/{instance_id}/{scene_id}` . . . 20
5.10.1 Method `Post` . . . 21
5.10.2 Content-Type `application/json` . . . 21
5.10.3 参数 . . . 21
5.10.4 返回 . . . 21
5.10.5 例子 . . . 21
5.11 接口 `/vision/service/build/{instance_id}` . . . 21
5.11.1 Method `Post` . . . 21
5.11.2 Content-Type `application/json` . . . 21
5.11.3 参数 . . . 21
5.11.4 返回 . . . 22
5.11.5 例子 . . . 22
5.12 接口 `/vision/service/autothreshold/{instance_id}` . . . 23
5.12.1 Method `Post` . . . 24
5.12.2 Content-Type `application/json` . . . 24
5.12.3 参数 . . . 24
5.12.4 返回 . . . 24
5.12.5 例子 . . . 24
5.13 接口 `/vision/service/query/{instance_id}` . . . 24
5.13.1 Method `Post` . . . 24

5.13.2 Content-Type `multipart/form-data` . . . 24
5.13.3 参数 . . . 24
5.13.4 返回 . . . 25
5.13.5 例子 . . . 25
5.14 接口 `/vision/scene/export/{instance_id}/{scene_id}` . . . 26
5.14.1 Method `Get` . . . 26
5.14.2 Content-Type `application/json` . . . 26
5.14.3 参数 . . . 26
5.14.4 返回 . . . 26
5.14.5 例子 . . . 27
5.15 接口 `/vision/scene/batch/export/{instance_id}` . . . 27
5.15.1 Method `Post` . . . 27
5.15.2 Content-Type `application/json` . . . 27
5.15.3 参数 . . . 27
5.15.4 返回 . . . 27
5.15.5 例子 . . . 28
5.16 接口 `/vision/scene/import/{instance_id}/{scene_id}` . . . 28
5.16.1 Method `Post` . . . 29
5.16.2 Content-Type `multipart/form-data` . . . 29
5.16.3 参数 . . . 29
5.16.4 返回 . . . 29
5.16.5 例子 . . . 29
5.17 接口 `/vision/scene/batch/import/{instance_id}` . . . 29
5.17.1 Method `Post` . . . 29
5.17.2 Content-Type `multipart/form-data` . . . 29
5.17.3 参数 . . . 29
5.17.4 返回 . . . 30
5.17.5 例子 . . . 30

6 知识库服务 30

6.1 接口 `/knowledge/service/enable` . . . 30
6.1.1 Method `Get` . . . 30
6.1.2 Content-Type `application/json` . . . 30
6.1.3 参数 . . . 30
6.1.4 返回 . . . 30
6.1.5 例子 . . . 31
6.2 接口 `/knowledge/service/enable/{enable}` . . . 31
6.2.1 Method `Post` . . . 31
6.2.2 Content-Type `application/json` . . . 31
6.2.3 参数 . . . 31
6.2.4 返回 . . . 31
6.2.5 例子 . . . 31
6.3 接口 `/knowledge/service/rerank/enable/{enable}` . . . 32
6.3.1 Method `Post` . . . 32

6.3.2 Content-Type `application/json` . . . 32
6.3.3 参数 . . . 32
6.3.4 返回 . . . 32
6.3.5 例子 . . . 32
6.4 接口 `/knowledge/service/splitter/enable/{enable}` . . . 32
6.4.1 Method `Post` . . . 32
6.4.2 Content-Type `application/json` . . . 32
6.4.3 参数 . . . 32
6.4.4 返回 . . . 32
6.4.5 例子 . . . 33
6.5 接口 `/knowledge/service/embedding/enable/{enable}` . . . 33
6.5.1 Method `Post` . . . 33
6.5.2 Content-Type `application/json` . . . 33
6.5.3 参数 . . . 33
6.5.4 返回 . . . 33
6.5.5 例子 . . . 33
6.6 接口 `/knowledge/service/llm/enable/{enable}` . . . 33
6.6.1 Method `Post` . . . 34
6.6.2 Content-Type `application/json` . . . 34
6.6.3 参数 . . . 34
6.6.4 返回 . . . 34
6.6.5 例子 . . . 34
6.7 接口 `/knowledge/service/init/{instance_id}` . . . 34
6.7.1 Method `Post` . . . 34
6.7.2 Content-Type `application/json` . . . 34
6.7.3 参数 . . . 34
6.7.4 返回 . . . 34
6.7.5 例子 . . . 34
6.8 接口 `/knowledge/service/list` . . . 35
6.8.1 Method `Get` . . . 35
6.8.2 Content-Type `application/json` . . . 35
6.8.3 参数 . . . 35
6.8.4 返回 . . . 35
6.8.5 例子 . . . 35
6.9 接口 `/knowledge/service/delete/{instance_id}` . . . 35
6.9.1 Method `Post` . . . 35
6.9.2 Content-Type `application/json` . . . 35
6.9.3 参数 . . . 35
6.9.4 返回 . . . 35
6.9.5 例子 . . . 36
6.10 接口 `/knowledge/knowledge/insert/{instance_id}/{knowledge_id}` . . . 36
6.10.1 Method `Post` . . . 36
6.10.2 Content-Type `multipart/form-data` . . . 36

6.10.3 参数 . . . 36
6.10.4 返回 . . . 36
6.10.5 例子 . . . 36
6.11 接口 `/knowledge/knowledge/list/{instance_id}` . . . 37
6.11.1 Method `Get` . . . 37
6.11.2 Content-Type `application/json` . . . 37
6.11.3 参数 . . . 37
6.11.4 返回 . . . 37
6.11.5 例子 . . . 37
6.12 接口 `/knowledge/knowledge/text/list/{instance_id}/{knowledge_id}` . . . 38
6.12.1 Method `GET` . . . 38
6.12.2 Content-Type `application/json` . . . 38
6.12.3 参数 . . . 38
6.12.4 返回 . . . 38
6.12.5 例子 . . . 38
6.13 接口 `/knowledge/knowledge/text/delete/{instance_id}/{knowledge_id}` . . . 39
6.13.1 Method `Post` . . . 39
6.13.2 Content-Type `application/json` . . . 39
6.13.3 参数 . . . 39
6.13.4 返回 . . . 39
6.13.5 例子 . . . 39
6.14 接口 `/knowledge/knowledge/delete/{instance_id}/{knowledge_id}` . . . 40
6.14.1 Method `Post` . . . 40
6.14.2 Content-Type `application/json` . . . 40
6.14.3 参数 . . . 40
6.14.4 返回 . . . 40
6.14.5 例子 . . . 40
6.15 接口 `/knowledge/service/build/{instance_id}` . . . 40
6.15.1 Method `Post` . . . 41
6.15.2 Content-Type `application/json` . . . 41
6.15.3 参数 . . . 41
6.15.4 返回 . . . 41
6.15.5 例子 . . . 41
6.16 接口 `/knowledge/service/query/{instance_id}` . . . 42
6.16.1 Method `Post` . . . 43
6.16.2 Content-Type `application/json` . . . 43
6.16.3 参数 . . . 43
6.16.4 返回 . . . 44
6.16.5 例子 . . . 45

7 ASR 服务 **51**
7.1 接口 `/asr/service/enable` . . . 51
7.1.1 Method `Get` . . . 51
7.1.2 Content-Type `application/json` . . . 51

7.1.3 参数 . . . 51
7.1.4 返回 . . . 51
7.1.5 例子 . . . 52
7.2 接口 /asr/service/enable/{enable} . . . 52
7.2.1 Method Post . . . 52
7.2.2 Content-Type application/json . . . 52
7.2.3 参数 . . . 52
7.2.4 返回 . . . 52
7.2.5 例子 . . . 52
7.3 接口 /asr/service/query/sample . . . 52
7.3.1 Method Post . . . 53
7.3.2 Content-Type application/json . . . 53
7.3.3 参数 . . . 53
7.3.4 返回 . . . 53
7.3.5 例子 . . . 53
7.4 接口 /asr/service/query/file . . . 54
7.4.1 Method Post . . . 54
7.4.2 Content-Type multipart/form-data . . . 54
7.4.3 参数 . . . 54
7.4.4 返回 . . . 54
7.4.5 例子 . . . 55

8 VLM 服务 56
8.1 接口 /vlm/service/enable . . . 56
8.1.1 Method Get . . . 56
8.1.2 Content-Type application/json . . . 56
8.1.3 参数 . . . 56
8.1.4 返回 . . . 56
8.1.5 例子 . . . 56
8.2 接口 /vlm/service/enable/{enable} . . . 56
8.2.1 Method Post . . . 56
8.2.2 Content-Type application/json . . . 56
8.2.3 参数 . . . 56
8.2.4 返回 . . . 57
8.2.5 例子 . . . 57
8.3 接口 /vlm/service/query . . . 57
8.3.1 Method Post . . . 57
8.3.2 Content-Type multipart/form-data . . . 57
8.3.3 参数 . . . 57
8.3.4 返回 . . . 57
8.3.5 例子 . . . 57

9 OCR 服务 59
9.1 接口 /ocr/service/enable . . . 59
9.1.1 Method Get . . . 59

9.1.2 Content-Type `application/json` . . . 59
9.1.3 参数 . . . 59
9.1.4 返回 . . . 59
9.1.5 例子 . . . 60
9.2 接口 `/ocr/service/enable/{enable}` . . . 60
9.2.1 Method `Post` . . . 60
9.2.2 Content-Type `application/json` . . . 60
9.2.3 参数 . . . 60
9.2.4 返回 . . . 60
9.2.5 例子 . . . 60
9.3 接口 `/ocr/service/query/file` . . . 60
9.3.1 Method `Post` . . . 60
9.3.2 Content-Type `multipart/form-data` . . . 60
9.3.3 参数 . . . 60
9.3.4 返回 . . . 61
9.3.5 例子 . . . 61

10 LLM 服务 **62**
10.1 接口 `/llm/service/enable` . . . 62
10.1.1 Method `Get` . . . 62
10.1.2 Content-Type `application/json` . . . 62
10.1.3 参数 . . . 62
10.1.4 返回 . . . 62
10.1.5 例子 . . . 62
10.2 接口 `/llm/service/enable/{enable}` . . . 63
10.2.1 Method `Post` . . . 63
10.2.2 Content-Type `application/json` . . . 63
10.2.3 参数 . . . 63
10.2.4 返回 . . . 63
10.2.5 例子 . . . 63
10.3 接口 `/llm/service/query` . . . 63
10.3.1 Method `Post` . . . 63
10.3.2 Content-Type `application/json` . . . 63
10.3.3 参数 . . . 63
10.3.4 返回 . . . 63
10.3.5 例子 . . . 64

11 OpenAI 兼容接口 **67**
11.1 接口 `/v1/chat/completions` . . . 68
11.1.1 Method `Post` . . . 68
11.1.2 Content-Type `application/json` . . . 68
11.1.3 参数 . . . 68
11.1.4 消息格式 . . . 68
11.1.5 返回（非流式） . . . 69
11.1.6 返回（流式） . . . 69
11.1.7 错误返回 . . . 69

11.1.8 例子 . . . 69
11.2 接口 /v1/embeddings . . . 71
11.2.1 Method Post . . . 71
11.2.2 Content-Type application/json . . . 71
11.2.3 参数 . . . 71
11.2.4 返回 . . . 71
11.2.5 例子 . . . 72
11.3 接口 /v1/audio/transcriptions . . . 72
11.3.1 Method Post . . . 72
11.3.2 Content-Type multipart/form-data . . . 72
11.3.3 参数 . . . 72
11.3.4 返回 . . . 72
11.3.5 例子 . . . 72
11.4 接口 /v1/models . . . 73
11.4.1 Method Get . . . 73
11.4.2 Content-Type application/json . . . 73
11.4.3 参数 . . . 73
11.4.4 返回 . . . 73
11.4.5 例子 . . . 73
11.5 流式返回说明 . . . 73
11.6 限制说明 . . . 73

1 游戏助手服务接口说明

编辑日期 2026-08-17 。
游戏助手服务版本 2026-08-17 。

2 更新日志

2.1 2026-08-13

- OpenAI 兼容接口 的 /v1/chat/completions 在消息包含图片时会透传 temperature 、 top_p 、 max_tokens 、 repetition_penalty 、 enable_thinking 等生成参数给 VLM 服务。
- VLM 增加 enable_thinking 参数，默认关闭思考模式，可通过请求参数或 vlm.json 配置开启。

2.2 2026-07-31

- 增加 OpenAI 兼容接口 ，支持以下 OpenAI 标准接口，可直接使用 OpenAI SDK 调用。
  - /v1/chat/completions
  - /v1/embeddings
  - /v1/audio/transcriptions
  - /v1/models

2.3 2025-09-23

- 知识库服务 的 query 接口增加 text_ids 返回说明。

2.4 2025-08-29

- 更新 VLM 服务 参数，支持传入多张图片。

2.5 2025-08-11

- `Vision 服务` 的 `insert` 接口返回 `invalid_pictures` 无法解析的图片列表。

2.6 2025-08-06

- 增加 `LLM 服务` 。

2.7 2025-08-05

- 增加 `OCR 服务` 。

2.8 2025-08-04

- 增加 `VLM 服务` 。

2.9 2025-07-25

- 增加 `关闭服务` `强制结束服务进程接口` 。

2.10 2025-06-26

- `知识库服务` 的 `build` 接口将默认以增量方式构建，同时增加参数 `full_build` ，可以全量方式重新构建。

2.11 2025-06-25

- `知识库服务` 的 `query` 接口增加 `system_message` 和 `prompt_template` 参数。

2.12 2025-06-24

- `Vision 服务` 的 `build` 接口增加 `auto_threshold` 参数。
- 增加 `Vision 服务` `自动阈值` 。

2.13 2025-06-06

- `Vision 服务` 的 `insert` 接口增加 `mask_file` 参数。

2.14 2025-05-30

- `Vision 服务` 的 `query` 接口增加 `mask_file` 参数。

2.15 2025-05-12

- `Vision 服务` 的 `build` 接口增加 `build_error` 事件名。
- `Vision 服务` 的 `删除场景图片` 接口增加返回内容。
- `知识库服务` 的 `删除知识库文本` 接口增加返回内容。

2.16 2025-05-09

- 增加 `关闭服务接口` 。
- 增加 `Vision 服务` `insert` 参数 `mode` 。
- 修改 `Vision 服务` `build` 和 `query` 参数 `mode` 的说明。

2.17 2025-04-29

- 修改 `Vision 服务` `导出场景接口` 的 `private-data-header` 中的 `sha1` 为 `sha256` 。
- 修改 `Vision 服务` `批量导出场景接口` 的 `private-data-header` 中的 `sha1` 为 `sha256` 。

2.18 2025-04-25

- `Vision 服务` 的 `build` 接口在 `build_finish` 时，返回推荐的 `threshold_2` 。
- `Vision 服务` 的 `query` 接口增加参数 `threshold_2` 。

2.19 2025-04-23

- 修改 `Vision 服务` `批量导出场景接口` 的 Method 为 Post。
- 增加 `Vision 服务` `build` 和 `query` 参数 `mode` 。

2.20 2025-04-18

- 增加 `Vision 服务` `批量导出场景接口` 。
- 增加 `Vision 服务` `批量导入场景接口` 。

2.21 2025-04-16

- 增加 `Vision 服务` `导出场景接口` 。
- 增加 `Vision 服务` `导入场景接口` 。

2.22 2025-03-17

- `知识库服务` 的 `build` 接口增加 `build_error` 事件名。

2.23 2025-03-04

- 增加 `配置文件` `重新加载的接口` 。
- 增加 `Vision 服务` `启停接口` 。
- 增加 `知识库服务` `启停接口` 。
- 增加 `ASR 服务` `启停接口` 。

2.24 2025-02-27

- `知识库服务` 的 `query` 接口查询过滤条件参数更新。

2.25 2025-02-18

- `Vision 服务` 的 `build` 接口在 `build_finish` 时，返回推荐的 `threshold` 。

2.26 2025-02-07

- 增加 `更新日志` 。

2.27 2025-01-27

- 简化 `ASR 服务` 的 `sample` 接口，不再需要传递数据类型。
- 知识库 `query` 接口增加额外信息参数，以实现自定义 Prompt 和查询知识库的能力。

2.28 2025-01-24

- `Vision 服务` 的 `build` 接口增加参数 `full_build` ，可以全量方式重新构建。
- `知识库服务` 的 `build` 接口增加单个文件的构建进度。

2.29 2025-01-23

- 增加 `ASR 服务` 接口说明。

2.30 2025-01-21

- 增加 `知识库服务` 接口说明。

2.31 2025-01-17

- 初版。

- 提供 `Vision 服务` 接口说明。

3 关闭服务

关闭服务的方式有两种。
1. 通过调用接口 `/server/shutdown` 来安全关闭服务。
2. 通过结束服务进程来关闭服务，但不建议在创建、插入、构建时强制关闭，以免造成数据库损坏和数据不一致的问题。

3.1 接口 `/server/shutdown`

发送关闭信号给服务，并立即返回，服务不会立即关闭，而是在所有连接断开后关闭。
需要注意的是，关闭信号一旦发送，将无法取消，也无法调用任何接口，任务执行中的接口不受影响。
建议在耗时较长的接口执行完成后，再调用该接口，可以立即关闭服务。

3.1.1 Method `Post`

3.1.2 Content-Type `application/json`

3.1.3 参数

无

3.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因"
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

3.1.5 例子

```
# 关闭服务

import requests

url = f"http://127.0.0.1:9190/server/shutdown"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

3.2 接口 `/server/shutdown_kill`

请谨慎使用此接口。推荐使用 `/server/shutdown` 来正常关闭服务。
调用此接口将强制结束服务进程，所有未保存的数据都会丢失，并且有概率损坏写入中的数据。

3.2.1 Method `Post`

3.2.2 Content-Type `application/json`

3.2.3 参数

无

3.2.4 返回

无

3.2.5 例子

```
# 强制结束服务进程

import requests

url = f"http://127.0.0.1:9190/server/shutdown_kill"
resp = requests.post(url)
```

4 配置文件

4.1 接口 `/config/reload`

重新读取配置文件并更新各个服务的配置项。
如果更换了模型文件路径或其它配置项，可以通过此接口使其生效，而无需重启整个服务。

4.1.1 Method `Post`

4.1.2 Content-Type `application/json`

4.1.3 参数

无

4.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因"
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

4.1.5 例子

```
# 重新加载配置文件

import requests

url = f"http://127.0.0.1:9190/config/reload"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5 Vision 服务

上传场景图片，构建场景数据库，提供场景识别功能。

5.1 接口 `/vision/service/enable`

查询 Vision 服务是否启用。

5.1.1 Method `Get`

5.1.2 Content-Type `application/json`

5.1.3 参数

无

5.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "vision": bool 类型。是否启用 Vision 服务。
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

5.1.5 例子

```
# 查询 Vision 服务是否启用

import requests

url = f"http://127.0.0.1:9190/vision/service/enable"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"vision": True}}
```

5.2 接口 `/vision/service/enable/{enable}`

启用或停用 Vision 服务。

5.2.1 Method `Post`

5.2.2 Content-Type `application/json`

5.2.3 参数

1. `enable` ：Bool 类型。`true` 启用，`false` 停用。

5.2.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` :
1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

5.2.5 例子

```
# 启用 Vision 服务实例

import requests

url = f"http://127.0.0.1:9190/vision/service/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5.3 接口 `/vision/service/init/{instance_id}`

创建 Vision 服务实例。

5.3.1 Method `Post`

5.3.2 Content-Type `application/json`

5.3.3 参数

1. `instance_id` : String 类型。Vision 实例 ID。

5.3.4 返回

格式:

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` :
1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

5.3.5 例子

```
# 创建 Vision 服务实例，实例 ID 为`test_instance`

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/vision/service/init/{instance_id}"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5.4 接口 `/vision/service/list`

列出所有 Vision 服务实例。

5.4.1 Method `Get`

5.4.2 Content-Type `application/json`

5.4.3 参数

无

5.4.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "instances_id": [
            "Vision 服务实例 ID 列表"
        ]
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

5.4.5 例子

```
# 列出所有 Vision 服务实例

import requests

url = f"http://127.0.0.1:9190/vision/service/list"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"instances_id": ["test_instance"]}}
```

5.5 接口 `/vision/service/delete/{instance_id}`

删除 Vision 服务实例。

5.5.1 Method `Post`

5.5.2 Content-Type `application/json`

5.5.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。

5.5.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

5.5.5 例子

```
# 删除 Vision 服务实例，实例名为`test_instance`

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/vision/service/delete/{instance_id}"
resp = requests.post(url)
```

```
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5.6 接口 /vision/scene/insert/{instance_id}/{scene_id}

向 Vision 服务实例添加一个场景。
可同时添加多张图片，以及对应的图片 ID，查询场景的结果中会返回图片 ID。
如果场景已存在，会追加图片。
如果图片 ID 与场景中图片 ID 一样，则会替换。

5.6.1 Method Post

5.6.2 Content-Type multipart/form-data

5.6.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。
2. `scene_id` ：String 类型。场景 ID。
3. `pictures` ：List[Byte] 类型。图片列表，列表中的每个元素是文件的 Byte 数据。
4. `pictures_id` ：List[String] 类型。图片 ID 列表，列表中的每个元素是图片 ID，索引位置需要和 `pictures` 对应。
5. `mask_file` ：Byte 类型。Mask 图片文件的 Byte 数据。可以不传，即不使用 Mask 图片。Mask 图片中的纯白色像素点作为擦除点，其它颜色像素点作为保留点。
6. `mode` ：String 类型。场景模式，有两种，`basic` 和 `accurate` ，默认 `accurate` 。
   `basic` 模式适用于 2D 场景，过场动画以及对时延有要求的场合（例如 Boss 招式识别）。
   `accurate` 模式适用于通用 3D 游戏场景标注，准确率更高，会比 `basic` 模式占用更多存储空间。

5.6.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "invalid_pictures": [
            {
                "name": " 无法解析的图片文件名",
                "size": 文件大小
            }
        ]
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。如果有无法解析的图片，会出现 `invalid_pictures` 列表。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

5.6.5 例子

```
# 添加场景，
# Vision 服务实例 ID 为`test_instance`，
# 场景 ID 为`test_scene`

import requests
```

```
pic_1 = r"C:\Users\Public\Pictures\场景 1_ 图片 1.jpg"
pic_2 = r"C:\Users\Public\Pictures\场景 1_ 图片 2.png"
mask = r"C:\Users\Public\Pictures\mask.png"

instance_id = "test_instance"
scene_id = "test_scene"
url = f"http://127.0.0.1:9190/vision/scene/insert/{instance_id}/{scene_id}"

files = [ ("pictures", open(pic_1, "rb")), ("pictures", open(pic_2, "rb")) ]
files += [ ("mask_file", open(mask, "rb")) ]
data = { "pictures_id": [ "pic_id_1", "pic_id_2" ], "mode": "accurate" }

resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5.7 接口 /vision/scene/list/{instance_id}

列出指定 Vision 服务实例下的所有场景 ID。

5.7.1 Method Get

5.7.2 Content-Type application/json

5.7.3 参数

1. instance_id ：String 类型。Vision 实例 ID。

5.7.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "scenes_id": [
            " 场景 ID 列表"
        ]
    }
}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

5.7.5 例子

```
# 列出所有场景 ID,
# Vision 服务实例 ID 为`test_instance`。

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/vision/scene/list/{instance_id}"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"scenes_id": ["test_scene"]}}}
```

5.8 接口 /vision/scene/picture/list/{instance_id}/{scene_id}

列出指定的场景中的图片 ID。

5.8.1 Method GET

5.8.2 Content-Type application/json

5.8.3 参数

1. instance_id ：String 类型。Vision 实例 ID。
2. scene_id ：String 类型。场景 ID。

5.8.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "pictures_id": [
            " 图片 ID 列表"
        ]
    }
}
```

返回的 code ：

1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

5.8.5 例子

```
# 列出场景图片 ID,
# Vision 服务实例 ID 为`test_instance`,
# 场景 ID 为`test_scene`

import requests

instance_id = "test_instance"
scene_id = "test_scene"
url = f"http://127.0.0.1:9190/vision/scene/picture/list/{instance_id}/{scene_id}"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"pictures_id": ["pic_id_1", "pic_id_2"]}}
```

5.9 接口 /vision/scene/picture/delete/{instance_id}/{scene_id}

从指定的场景中删除图片。

5.9.1 Method Post

5.9.2 Content-Type application/json

5.9.3 参数

1. instance_id ：String 类型。Vision 实例 ID。
2. scene_id ：String 类型。场景 ID。

3. `pictures_id` : List[String] 类型。图片 ID 列表。

5.9.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "deleted": [
            " 成功删除的图片 ID 列表"
        ],
        "not_exist": [
            " 不存在的图片 ID 列表"
        ]
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

5.9.5 例子

```
# 删除场景图片，
# Vision 服务实例 ID 为`test_instance`，
# 场景 ID 为`test_scene`，
# 图片 ID 为`pic_id_1`和`pic_id_2`，
# 以及不存在的图片 ID 为`not_exists_id_1`和`not_exists_id_2`。

import json
import requests

instance_id = "test_instance"
scene_id = "test_scene"
url = f"http://127.0.0.1:9190/vision/scene/picture/delete/{instance_id}/{scene_id}"
data = { "pictures_id": [ "pic_id_1", "pic_id_2", "not_exists_id_1", "not_exists_id_2" ] }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data:
{
    "code": "ok",
    "data": {
        "deleted": [ "pic_id_1", "pic_id_2" ],
        "not_exist": [ "not_exists_id_1", "not_exists_id_2" ]
    }
}
"""
```

5.10 接口 `/vision/scene/delete/{instance_id}/{scene_id}`

从 Vision 服务实例删除一个场景。

5.10.1 Method `Post`

5.10.2 Content-Type `application/json`

5.10.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。
2. `scene_id` ：String 类型。场景 ID。

5.10.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

5.10.5 例子

```
# 删除场景，
# Vision 服务实例 ID 为`test_instance`，
# 场景 ID 为`test_scene`

import requests

instance_id = "test_instance"
scene_id = "test_scene"
url = f"http://127.0.0.1:9190/vision/scene/delete/{instance_id}/{scene_id}"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5.11 接口 `/vision/service/build/{instance_id}`

构建 Vision 服务实例数据库。
构建过程时长取决于新图片的数量和机器性能，已经构建过的图片不会重新构建。
如果出现无法识别的图片，会跳过该文件继续构建。
以流式返回构建进度。

5.11.1 Method `Post`

5.11.2 Content-Type `application/json`

5.11.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。
2. `mode` ：String 类型。构建模式，有两种，`basic` 和 `accurate` ，默认 `accurate` 。
   如果以 `basic` 模式构建，所有 `accurate` 的场景都会变成 `basic` 模式。
   如果以 `accurate` 模式构建，场景以各自的模式构建。
3. `full_build` ：Bool 类型。`true` 全新构建，默认 `false` 增量构建。当增删改较多的时候，或者发现查询结果和预期的不一致，建议做一次全新构建，以消除隐藏的 BUG 导致的问题。
4. `auto_threshold` ：Bool 类型。`true` 表示计算自动阈值，`false` 表示不计算自动阈值，默认为 `true` 。如果返回的阈值为 `-1` ，表示使用配置文件中设定的阈值。

5.11.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "event_name": " 构建事件名",
        "total": int 类型，总构建图片数量，
        "current": int 类型，当前构建图片索引，从1开始，
        "picture_id": string 类型，" 当前构建图片 ID",
        "threshold": float 类型，推荐的阈值，在调用查询接口时传入，
        "threshold_2": float 类型，推荐的阈值2，在调用查询接口时传入，仅在`mode`为`accurate`时生效
    },
    "created_time": float 类型，事件创建时间（秒），
    "fin": bool 类型，流式返回是否结束
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

`event_name` 说明：

1. `build_init` ：构建开始。
2. `build_progress` ：构建进度。`total` 、`current` 、`picture_id` 仅在此阶段出现。
3. `build_finish` ：构建完成。`threshold` 和 `threshold_2` 仅在此阶段出现。
4. `build_error` ：构建错误。通常是由于无法解析图片文件引起的。该错误仅针对单个文件，不会中断整个构建过程。

5.11.5 例子

```
# 构建场景，Vision 服务实例 ID 为`test_instance`。
# 流式返回构建进度。

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/vision/service/build/{instance_id}"
headers = { "Accept": "text/event-stream" }
data = json.dumps({"mode": "accurate", "full_build": false, "auto_threshold": true})
resp = requests.post(url, headers=headers, data=data, stream=True)

for chunk in resp.iter_content(chunk_size=1024):
    print(chunk.decode("utf-8"))

"""
data: {
    "code": "ok",
    "data": {
        "event_name": "build_init"
    },
    "created_time": 1737081488.7292595,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
```

```
        "event_name": "build_progress",
        "total": 3,
        "current": 1,
        "picture_id": "pic_id_1"
    },
    "created_time": 1737081488.8790212,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "event_name": "build_progress",
        "total": 3,
        "current": 2,
        "picture_id": "pic_id_2"
    },
    "created_time": 1737081488.9990811,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "event_name": "build_error",
        "total": 3,
        "current": 3,
        "picture_id": "not_a_picture"
    },
    "created_time": 1737081489.1190253,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "event_name": "build_finish",
        "threshold": 0.92,
        "threshold_2": 0.15
    },
    "created_time": 1737081489.0030932,
    "fin": true
}


"""
```

5.12 接口 `/vision/service/autothreshold/{instance_id}`

计算 Vision 服务实例的自动阈值。
计算时长取决于图片的数量和机器性能。
如果返回的阈值为 `-1` ，表示使用配置文件中设定的阈值。

5.12.1 Method `Post`

5.12.2 Content-Type `application/json`

5.12.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。

5.12.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "threshold": float 类型，推荐的阈值，在调用查询接口时传入，
        "threshold_2": float 类型，推荐的阈值2，在调用查询接口时传入，仅在`mode`为`accurate`时生效
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

5.12.5 例子

```
# 计算自动阈值，Vision 服务实例 ID 为`test_instance`。

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/vision/service/autothreshold/{instance_id}"
resp = requests.post(url)
resp_data = resp.json()

""" resp_data:
{
    "code": "ok",
    "data": {
        "threshold": 0.92,
        "threshold_2": 0.15
    }
}
"""
```

5.13 接口 `/vision/service/query/{instance_id}`

查询 Vision 服务实例的场景数据库。
如果场景发生过修改、添加、删除，需要重新构建数据库才会使查询生效。

5.13.1 Method `Post`

5.13.2 Content-Type `multipart/form-data`

5.13.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。
2. `file` ：Byte 类型。图片文件的 Byte 数据。

3. `mask_file` ：Byte 类型。Mask 图片文件的 Byte 数据。可以不传，即不使用 Mask 图片。Mask 图片中的纯白色像素点作为擦除点，其它颜色像素点作为保留点。

4. `topk` ：Int 类型。仅返回分数最高的 K 个场景 ID。必须大于 0。

5. `threshold` ：Float 类型。仅返回分数超过 `threshold` 的场景。取值范围 `0.0 ~ 1.0` ，如果传入 `-1` 会替换为预设的阈值。

6. `threshold_2` ：Float 类型。仅在 `mode` 为 `accurate` 时生效。取值范围 `0.0 ~ 1.0` ，如果传入 `-1` 会替换为预设的阈值。

7. `mode` ：String 类型。查询模式，同构建模式一样有两种， `basic` 和 `accurate` ，默认 `accurate` 。
   如果以 `basic` 模式查询，所有 `accurate` 的场景都会以 `basic` 模式查询。
   如果以 `accurate` 模式查询，场景以各自的模式查询。
   当查询结果同时包含 `basic` 和 `accurate` 场景时，优先返回 `accurate` 场景。

5.13.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": [
        {
            "scene_id": " 场景 ID",
            "picture_id": " 图片 ID",
            "score": float 类型，范围`0.0 ~ 1.0`，分数越高越匹配
        }
    ]
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

5.13.5 例子

```
# 查询场景数据库，
# Vision 服务实例 ID 为`test_instance`，
# 场景 ID 为`test_scene`

import requests

pic = r"C:\Users\Public\Pictures\场景 1_ 图片 1.jpg"
mask = r"C:\Users\Public\Pictures\场景 1_ 图片 1_mask.png"

instance_id = "test_instance"
scene_id = "test_scene"
url = f"http://127.0.0.1:9190/vision/service/query/{instance_id}"

files = { "file": open(pic, "rb"), "mask_file": open(mask, "rb") }
data = { "topk": 2, "threshold": 0.618, "threshold_2": 0.15, "mode": "accurate" }

resp = requests.post(url, data=data, files=files)
resp_data = resp.json()

""" resp_data:
{
```

```
    "code": "ok",
    "data": [
        {
            "scene_id": "test_scene",
            "picture_id": "pic_id_1",
            "score": 0.9999998807907104
        },
        {
            "scene_id": "test_scene",
            "picture_id": "pic_id_2",
            "score": 0.8671993017196655
        }
    ]
}
"""
```

5.14 接口 /vision/scene/export/{instance_id}/{scene_id}

导出 Vision 服务实例的场景数据。

5.14.1 Method Get

5.14.2 Content-Type application/json

5.14.3 参数

1. instance_id ：String 类型。Vision 实例 ID。
2. scene_id ：String 类型。场景 ID。

5.14.4 返回

1. 返回 PrivateData 二进制数据流。
2. 返回 PrivateData 的 Header，以 json 字符串的形式存于 Response 的 Headers 中，键名为 private-data-header 。

PrivateData 的 Header 格式：

```
{
    "version": "1.0",
    "instance_id": "test_instance",
    "scene_id": "test_scene",
    "pictures": [
        "picture_1_id",
        "picture_2_id",
        "picture_3_id",
        "picture_N_id"
    ],
    "size": 12345678,
    "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
}
```

PrivateData 的 Header 字段说明：

1. version ：String 类型。PrivateData 的版本。
2. instance_id ：String 类型。PrivateData 对应的 instance_id 。
3. scene_id ：String 类型。PrivateData 对应的 scene_id 。

4. `pictures` ：List[String] 类型。PrivateData 包含的所有图片 ID。
5. `size` ：Int 类型。PrivateData 文件大小。
6. `sha256` ：String 类型。PrivateData 数据的 SHA256 哈希值，用于校验数据完整性。

5.14.5 例子

```
# 导出场景数据，
# Vision 服务实例 ID 为`test_instance`，
# 场景 ID 为`test_scene`

import requests

instance_id = "test_instance"
scene_id = "test_scene"
url = f"http://127.0.0.1:9190/vision/scene/export/{instance_id}/{scene_id}"

resp = requests.get(url)
header = resp.headers['private-data-header'] # PrivateDataHeader
content = resp.content # PrivateData 二进制数据

""" header:
{
    "version": "1.0",
    "instance_id": "test_instance",
    "scene_id": "test_scene",
    "pictures": [
        "pic_id_1",
        "pic_id_2"
    ],
    "size": 240,
    "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
}
"""
```

5.15 接口 `/vision/scene/batch/export/{instance_id}`

批量导出 Vision 服务实例的场景数据。

5.15.1 Method `Post`

5.15.2 Content-Type `application/json`

5.15.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。
2. `scenes_id` ：List[String] 类型。场景 ID 列表。

5.15.4 返回

1. 返回各个场景的 PrivateData 的 `header.json` 和 `data.dat` 打包后的 Zip 文件。

Zip 包内文件列表示例：

```
scene_1_data.dat
scene_1_header.json
scene_2_data.dat
scene_2_header.json
```

```
scene_3_data.dat
scene_3_header.json
scene_N_data.dat
scene_N_header.json
```

PrivateData 的 `header.json` 示例：

```
{
    "version": "1.0",
    "instance_id": "test_instance",
    "scene_id": "test_scene",
    "pictures": [
        "pic_id_1",
        "pic_id_2"
    ],
    "size": 12345678,
    "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
}
```

PrivateData 的 Header 字段说明：

1. `version` ：String 类型。PrivateData 的版本。
2. `instance_id` ：String 类型。PrivateData 对应的 `instance_id` 。
3. `scene_id` ：String 类型。PrivateData 对应的 `scene_id` 。
4. `pictures` ：List[String] 类型。PrivateData 包含的所有图片 ID。
5. `size` ：Int 类型。PrivateData 文件大小。
6. `sha256` ：String 类型。PrivateData 数据的 SHA256 哈希值，用于校验数据完整性。

5.15.5 例子

```
# 批量导出场景数据，
# Vision 服务实例 ID 为 `test_instance`，

import json
import requests

instance_id = "test_instance"

url = f"http://127.0.0.1:9190/vision/scene/batch/export/{instance_id}"

data = { "scenes_id": [ "test_scene_1", "test_scene_2" ] }

resp = requests.post(url, data=json.dumps(data))
content = resp.content # Zip 包
```

5.16 接口 `/vision/scene/import/{instance_id}/{scene_id}`

导入 Vision 服务实例的场景数据。

5.16.1 Method `Post`

5.16.2 Content-Type `multipart/form-data`

5.16.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。可以和 PrivateData 导出时的实例 ID 不同。
2. `scene_id` ：String 类型。场景 ID。可以和 PrivateData 导出时的场景 ID 不同。

5.16.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因"
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

5.16.5 例子

```
# 导入场景数据，
# Vision 服务实例 ID 为`test_instance`，
# 场景 ID 为`test_scene`

import requests

private_data_file = r"C:\Users\Public\Downloads\private_data.dat"

instance_id = "test_instance"
scene_id = "test_scene"
url = f"http://127.0.0.1:9190/vision/scene/import/{instance_id}/{scene_id}"

files = { "file": open(private_data_file, "rb") }

resp = requests.post(url, files=files)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5.17 接口 `/vision/scene/batch/import/{instance_id}`

批量导入 Vision 服务实例的场景数据。

5.17.1 Method `Post`

5.17.2 Content-Type `multipart/form-data`

5.17.3 参数

1. `instance_id` ：String 类型。Vision 实例 ID。可以和 PrivateData 导出时的实例 ID 不同。
2. `files` ：List[Byte] 类型。PrivateData 列表，列表中的每个元素是文件的 Byte 数据。
3. `scenes_id` ：List[String] 类型。场景 ID 列表，列表中的每个元素是场景 ID，索引位置需要和 `files` 对应，可以和 PrivateData 导出时的场景 ID 不同。

5.17.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因"
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

5.17.5 例子

```
# 批量导入场景数据，
# Vision 服务实例 ID 为`test_instance`，
# 场景 ID 为`test_scene`

import requests

file_1 = r"C:\Users\Public\Downloads\private_data_1.dat"
file_2 = r"C:\Users\Public\Downloads\private_data_2.dat"

instance_id = "test_instance"

url = f"http://127.0.0.1:9190/vision/scene/batch/import/{instance_id}"

files = [ ("files", open(file_1, "rb")), ("files", open(file_2, "rb")) ]
data = { "scenes_id": [ "test_scene_1", "test_scene_2" ] }

resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6 知识库服务

上传攻略文本，构建知识库，提供根据攻略回答问题的功能。

6.1 接口 `/knowledge/service/enable`

查询知识库服务是否启用。

6.1.1 Method `Get`

6.1.2 Content-Type `application/json`

6.1.3 参数

无

6.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "knowledge": bool 类型。是否启用知识库服务，
```

```
        "embedding": bool 类型。是否启用知识库的 Embedding 服务,
        "splitter": bool 类型。是否启用知识库的 Splitter 服务,
        "rerank": bool 类型。是否启用知识库的 Rerank 服务,
        "llm": bool 类型。是否启用知识库的 LLM 服务,
    }
}
```

返回的 `code` :
1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

6.1.5 例子

```
# 查询知识库服务是否启用

import requests

url = f"http://127.0.0.1:9190/knowledge/service/enable"
resp = requests.get(url)
resp_data = resp.json()
# resp_data:
{
    "code": "ok",
    "data": {
        "knowledge": true,
        "embedding": true,
        "splitter": true,
        "rerank": true,
        "llm": true
    }
}
```

6.2 接口 `/knowledge/service/enable/{enable}`

启用或停用知识库服务。

6.2.1 Method `Post`

6.2.2 Content-Type `application/json`

6.2.3 参数

1. `enable` : Bool 类型。`true` 启用，`false` 停用。

6.2.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` :
1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

6.2.5 例子

```
# 启用知识库服务实例

import requests
```

```
url = f"http://127.0.0.1:9190/knowledge/service/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.3 接口 /knowledge/service/rerank/enable/{enable}

启用或停用知识库的 Rerank 服务。
停用 Rerank 服务后，调用 query 接口时，将不再对检索到的知识进行重新打分和排序。

6.3.1 Method Post

6.3.2 Content-Type application/json

6.3.3 参数

1. enable ：Bool 类型。true 启用，false 停用。

6.3.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

6.3.5 例子

```
# 启用知识库的 Rerank 服务

import requests

url = f"http://127.0.0.1:9190/knowledge/service/rerank/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.4 接口 /knowledge/service/splitter/enable/{enable}

启用或停用知识库的 Splitter 服务。
停用 Splitter 服务会无法使用 build 接口。

6.4.1 Method Post

6.4.2 Content-Type application/json

6.4.3 参数

1. enable ：Bool 类型。true 启用，false 停用。

6.4.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

6.4.5 例子

```
# 启用知识库的 Splitter 服务

import requests

url = f"http://127.0.0.1:9190/knowledge/service/splitter/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.5 接口 /knowledge/service/embedding/enable/{enable}

启用或停用知识库的 Embedding 服务。
停用 Embedding 服务会无法使用 build 和 query 接口。

6.5.1 Method Post

6.5.2 Content-Type application/json

6.5.3 参数

1. enable ：Bool 类型。true 启用，false 停用。

6.5.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

6.5.5 例子

```
# 启用知识库的 Embedding 服务

import requests

url = f"http://127.0.0.1:9190/knowledge/service/embedding/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.6 接口 /knowledge/service/llm/enable/{enable}

启用或停用知识库的 LLM 服务。
停用 LLM 服务会无法使用 query 接口。

6.6.1 Method `Post`

6.6.2 Content-Type `application/json`

6.6.3 参数

1. `enable` ：Bool 类型。`true` 启用，`false` 停用。

6.6.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

6.6.5 例子

```
# 启用知识库的 LLM 服务

import requests

url = f"http://127.0.0.1:9190/knowledge/service/llm/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.7 接口 `/knowledge/service/init/{instance_id}`

创建知识库服务实例。

6.7.1 Method `Post`

6.7.2 Content-Type `application/json`

6.7.3 参数

1. `instance_id` ：String 类型。知识库实例 ID。

6.7.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

6.7.5 例子

```
# 创建知识库服务实例，实例 ID 为`test_instance`

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/knowledge/service/init/{instance_id}"
resp = requests.post(url)
```

```
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.8 接口 /knowledge/service/list

列出所有知识库服务实例。

6.8.1 Method Get

6.8.2 Content-Type application/json

6.8.3 参数

无

6.8.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "instances_id": [
            " 知识库服务实例 ID 列表"
        ]
    }
}
```

返回的 code ：

1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

6.8.5 例子

```
# 列出所有知识库服务实例

import requests

url = f"http://127.0.0.1:9190/knowledge/service/list"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"instances_id": ["test_instance"]}}
```

6.9 接口 /knowledge/service/delete/{instance_id}

删除知识库服务实例。

6.9.1 Method Post

6.9.2 Content-Type application/json

6.9.3 参数

1. instance_id ：String 类型。知识库实例 ID。

6.9.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

6.9.5 例子

```
# 删除知识库服务实例，实例名为`test_instance`

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/knowledge/service/delete/{instance_id}"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.10 接口 `/knowledge/knowledge/insert/{instance_id}/{knowledge_id}`

向知识库服务实例添加一个攻略。
可同时添加多个攻略文本文件，以及对应的文本 ID 和元数据，查询攻略的结果中会返回文本 ID。
如果攻略已存在，会追加文本。
如果文本 ID 与攻略中文本 ID 一样，则会替换。

6.10.1 Method `Post`

6.10.2 Content-Type `multipart/form-data`

6.10.3 参数

1. `instance_id` ：String 类型。知识库实例 ID。
2. `knowledge_id` ：String 类型。攻略 ID。
3. `texts` ：List[Byte] 类型。攻略文本列表，列表中的每个元素是文件的 Byte 数据。
4. `texts_id` ：List[String] 类型。文本 ID 列表，列表中的每个元素是文本 ID，索引位置需要和 `texts` 对应。
5. `texts_metadata` ：String 类型。JSON 格式的攻略元数据。以 `text_id` 作为键，其值是 Dict 类型，包含具体的元数据内容。如果某个文本没有元数据，该文本 `text_id` 对应的键值对可以留空或不填。
   可用的元数据如下：
   - scenes_name：List[String] 类型。场景名称。查询时传入场景名，将会在场景名对应的攻略中查找内容。

6.10.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

6.10.5 例子

```
# 添加攻略，
# 知识库服务实例 ID 为`test_instance`，
```

```
# 攻略 ID 为`test_knowledge`

import requests

text_1 = r"C:\Users\Public\Documents\Game_BossA.txt"
text_2 = r"C:\Users\Public\Documents\Game_BossB.txt"

instance_id = "test_instance"
knowledge_id = "test_knowledge"
url = f"http://127.0.0.1:9190/knowledge/knowledge/insert/{instance_id}/{knowledge_id}"

files = [ ("texts", open(text_1, "rb")), ("texts", open(text_2, "rb")) ]
data = {
    "texts_id": [ "text_id_1", "text_id_2" ],
    "texts_metadata": json.dumps(
        {
            "text_id_1": { "scenes_name": [ "Scene_1", "Scene_3" ] },
            "text_id_2": { "scenes_name": [ "Scene_2" ] }
        }
    )
}

resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.11 接口 /knowledge/knowledge/list/{instance_id}

列出指定知识库服务实例下的所有攻略 ID。

6.11.1 Method Get

6.11.2 Content-Type application/json

6.11.3 参数

1. instance_id ：String 类型。知识库实例 ID。

6.11.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "knowledge_ids": [
            " 攻略 ID 列表"
        ]
    }
}
```

返回的 code ：

1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

6.11.5 例子

```
# 列出所有攻略 ID,
# 知识库服务实例 ID 为`test_instance`。

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/knowledge/knowledge/list/{instance_id}"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"knowledge_ids": ["test_knowledge"]}}
```

6.12 接口 /knowledge/knowledge/text/list/{instance_id}/{knowledge_id}

列出指定的攻略中的文本 ID。

6.12.1 Method GET

6.12.2 Content-Type application/json

6.12.3 参数

1. instance_id ：String 类型。知识库实例 ID。
2. knowledge_id ：String 类型。攻略 ID。

6.12.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "texts_id": [
            " 图片 ID 列表"
        ]
    }
}
```

返回的 code ：

1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

6.12.5 例子

```
# 列出攻略文本 ID,
# 知识库服务实例 ID 为`test_instance`,
# 攻略 ID 为`test_knowledge`

import requests

instance_id = "test_instance"
knowledge_id = "test_knowledge"
url = f"http://127.0.0.1:9190/knowledge/knowledge/text/list/{instance_id}/{knowledge_id}"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"texts_id": ["text_id_1", "text_id_2"]}}
```

6.13 接口 /knowledge/knowledge/text/delete/{instance_id}/{knowledge_id}

从指定的攻略中删除文本。

6.13.1 Method Post

6.13.2 Content-Type application/json

6.13.3 参数

1. instance_id : String 类型。知识库实例 ID。
2. knowledge_id : String 类型。攻略 ID。
3. texts_id : List[String] 类型。文本 ID 列表。

6.13.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "deleted": [
            " 成功删除的图片 ID 列表"
        ],
        "not_exist": [
            " 不存在的图片 ID 列表"
        ]
    }
}
```

返回的 code :

1. ok : 成功。不会出现 msg 。
2. fail : 失败。会有 msg 给出失败原因。不会出现 data 。

6.13.5 例子

```
# 删除攻略文本,
# 知识库服务实例 ID 为`test_instance`,
# 攻略 ID 为`test_knowledge`,
# 文本 ID 为`text_id_1`和`text_id_2`,
# 以及不存在的文本 ID 为`not_exists_id_1`和`not_exists_id_2`。

import json
import requests

instance_id = "test_instance"
knowledge_id = "test_knowledge"
url = f"http://127.0.0.1:9190/knowledge/knowledge/text/delete/{instance_id}/{knowledge_id}"
data = { "texts_id": [ "text_id_1", "text_id_2", "not_exists_id_1", "not_exists_id_2" ] }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data:
{
    "code": "ok",
    "data": {
        "deleted": [ "text_id_1", "text_id_2" ],
        "not_exist": [ "not_exists_id_1", "not_exists_id_2" ]
```

```
    }
}
"""
```

6.14 接口 /knowledge/knowledge/delete/{instance_id}/{knowledge_id}

从知识库服务实例删除一个攻略。

6.14.1 Method Post

6.14.2 Content-Type application/json

6.14.3 参数

1. instance_id ：String 类型。知识库实例 ID。
2. knowledge_id ：String 类型。攻略 ID。

6.14.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

6.14.5 例子

```
# 删除攻略，
# 知识库服务实例 ID 为`test_instance`，
# 攻略 ID 为`test_knowledge`

import requests

instance_id = "test_instance"
knowledge_id = "test_knowledge"
url = f"http://127.0.0.1:9190/knowledge/knowledge/delete/{instance_id}/{knowledge_id}"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.15 接口 /knowledge/service/build/{instance_id}

构建知识库服务实例数据库。
构建过程时长取决于文本的数量和机器性能，已经构建过的文本不会重新构建。
以流式返回构建进度。

把一个大文件拆分为多个小文件不会加快构建速度。
以《西游记》为例，单个文件大小约为 2MB，5500 行，大约 70 万的文字量，使用 Ultra 7 265K 核显处理完成需要 10 分钟，使用 Arc A770 独显处理完成需要 1 分钟。
如果把《西游记》拆分成 100 个文件，使用 Ultra 7 265K 核显处理完成需要 11.5 分钟，使用 Arc A770 独显处理完成需要 1.1 分钟。

6.15.1 Method `Post`

6.15.2 Content-Type `application/json`

6.15.3 参数

1. `instance_id` ：String 类型。知识库实例 ID。
2. `full_build` ：Bool 类型。`true` 全新构建，默认 `false` 增量构建。当增删改较多的时候，或者发现查询结果和预期的不一致，建议做一次全新构建，以消除隐藏的 BUG 导致的问题。

6.15.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "event_name": " 构建事件名",
        "total": int 类型，总构建文本数量，
        "current": int 类型，当前构建文本索引，从1开始，
        "percent": int 类型，当前构建文本进度百分比，
        "text_id": " 当前构建文本 ID"
    },
    "created_time": float 类型，事件创建时间（秒），
    "fin": bool 类型，流式返回是否结束
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

`event_name` 说明：

1. `build_init` ：构建开始。
2. `build_progress` ：构建进度。`total` 、`current` 、`percent` 、`text_id` 仅在此阶段出现。
3. `build_finish` ：构建完成。
4. `build_error` ：构建错误。通常是由于文本编码不是 UTF-8 或者不是文本文件格式引起的。该错误仅针对单个文件，不会中断整个构建过程。

6.15.5 例子

```
# 构建场景，知识库服务实例 ID 为`test_instance`。
# 流式返回构建进度。

import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/knowledge/service/build/{instance_id}"
headers = { "Accept": "text/event-stream" }
data = json.dumps({'full_build': False})
resp = requests.post(url, headers=headers, data=data, stream=True)

for chunk in resp.iter_content(chunk_size=1024):
    print(chunk.decode("utf-8"))

"""
data: {
    "code": "ok",
```

```
    "data": {
        "event_name": "build_init"
    },
    "created_time": 1737445665.1809583,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "event_name": "build_progress",
        "total": 2,
        "current": 1,
        "percent": 100,
        "text_id": "text_id_1"
    },
    "created_time": 1737445666.0533168,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "event_name": "build_progress",
        "total": 2,
        "current": 2,
        "percent": 100,
        "text_id": "text_id_2"
    },
    "created_time": 1737445666.7595007,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "event_name": "build_finish"
    },
    "created_time": 1737445666.7756412,
    "fin": true
}


"""
```

6.16 接口 /knowledge/service/query/{instance_id}

查询知识库服务实例的攻略数据库。
如果文本发生过修改、添加、删除，需要重新构建数据库才会使查询生效。
该接口并非直接返回数据库内容，而是经由大语言模型总结后返回结果。

6.16.1 Method `Post`

6.16.2 Content-Type `application/json`

6.16.3 参数

1. `instance_id` ：String 类型。`必须` 。知识库实例 ID。

2. `text` ：String 类型。`必须` 。以自然语言的形式提出的问题。

3. `knowledge_id` ：List[String] 类型。`可选` 。攻略 ID 列表，仅从指定的攻略集合中查询。

4. `knowledge_id_op` ：String 类型。`可选` 。默认值为 `or` 。将指定的攻略集合与已过滤的结果做 `并集（or）` 、`交集（and）` 、`差集（diff）` 操作。

5. `texts_id` ：List[String] 类型。`可选` 。文本 ID 列表，仅从指定的文本集合中查询。

6. `texts_id_op` ：String 类型。`可选` 。默认值为 `or` 。将指定的文本集合与已过滤的结果做 `并集（or）` 、`交集（and）` 、`差集（diff）` 操作。

7. `scenes_name` ：List[String] 类型。`可选` 。场景名列表，仅从指定的场景对应的文本集合中查询。

8. `scenes_name_op` ：String 类型。`可选` 。默认值为 `orc` 。将指定的场景对应的文本集合与已过滤的结果做 `公共知识库并集（orc）` 、`并集（or）` 、`交集（and）` 、`差集（diff）` 操作。
   `公共知识库` ：不属于任何场景的知识库称作 `公共知识库` 。
   `公共知识库并集（orc）` ：考虑到在某个场景中，用户会询问一些非场景相关的问题，如果仅查询该场景名对应的知识库，可能无法给出回答。所以提供了 `orc` 选项，除了查询该场景名对应的知识库外，还会查询公共知识库。
   `过滤条件顺序` ：`knowledge_id` -> `texts_id` -> `scenes_name` ，前后结果以指定 `op` 做集合操作，比如取所有结果的交集 `knowledge_id & texts_id & scenes_name` 。
   如果过滤结果为空，则不查询数据库。

9. `extra_info` ：Dict[String, Dict[String, String]] 类型。`可选` 。额外查询信息。

   - `vsearch` ：String 类型。`可选` 。额外搜索数据库的关键词，搜索到的结果会追加到总的 Prompt 中，对回答产生影响。
   - `prompt` ：String 类型。`必须` 。自定义 Prompt，该 Prompt 里以 `$` 开头的变量会被替换成对应的值，然后添加到总的 Prompt 中。其它未使用的键会被忽略。
   - 格式：

```
{
    "extra_info": {
        "any_info_name": {
            "vsearch": " 此处的值被用于搜索数据库，其结果会对回答产生影响。",
            "prompt": " 键'$info_key_1'会被替换成'info_value_1'。",
            "info_key_1": "info_value_1",
            "info_key_2": "info_value_2",
            "info_key_N": "info_value_N",
        }
    }
}
```

10. `system_message` ：String 类型。`可选` 。如果不传，则使用服务端内置的 `system_message` 。如果传空字符串则不使用 `system_message` 。

注意：服务端会优先使用 < 服务端根目录 >\prompts\system_message.txt 文件内容作为 system_message 。内置的 system_message 为 你是 AI 游戏助手，解决游戏里玩家遇到的各种问题。你回答的内容应该控制在 200 字以内。。

11. prompt_template ：String 类型。 可选 。如果不传，则使用服务端内置的 提示词模板 。
注意：服务端会优先使用 < 服务端根目录 >\prompts\prompt_template.txt 文件内容作为 提示词模板 。
在模板中，可以使用 < 标签 > 内容 </标签 > 的形式将有特殊意义的内容括起来，方便模型理解，例如

```
< 人物设定 >
姓名：张三
年龄：233 岁
身高：23 公分
</人物设定 >
```

模板内置变量说明

- information ：经过知识库搜索到的内容以及 extra_info 对应的内容。
- user_message ：用户输入的内容。

12. threshold ：Float 类型。 可选 。重排序相关性门控阈值，覆盖配置文件中的 knowledge.query.threshold 。仅返回重排序分数不低于该值的知识块；若全部低于该值，接口直接返回 refusal_message 拒答消息（不调用 LLM）。

内置的 提示词模板 如下

```
< 指令 > 根据【资料】回答问题，遵守以下规则：
1. 只能使用【资料】中标注了编号（如 [1]）的内容作答，禁止编造资料中没有的信息。
2. 如果【资料】不足以回答，必须以“{refusal_message}”开头说明资料不足，并指出缺少什么，不要猜测。
3. 回答用中文，简洁有条理，控制在 200 字以内；需要时可以引用来源编号如 [1]。</指令 >
< 资料 >
{information}
</资料 >
< 问题 >{user_message}</问题 >
```

6.16.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "message": " 增量回答的内容",
        "text_ids": [" 查询到的知识库文本 ID"]
    },
    "created_time": float 类型，回答创建时间（秒），
    "fin": bool 类型，流式返回是否结束
}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

关于 text_ids ：
首次返回的 data 中会包含 text_ids ，即查询到的知识库文本 ID 列表。
若没有命中任何知识库内容，则会返回空列表。
首次返回是在完成查询知识库之后立即返回的，所以首次返回不包含 message ，
等语言模型吐出第一个字词开始，第二次返回才开始包含 message ，

因此在首次和第二次返回之间至少有 2 秒的时间间隔。
（模型大小、机器性能、提示词内容等因素会影响间隔时间）

6.16.5 例子

```
# 查询知识库，
# 知识库服务实例 ID 为`test_instance`

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/knowledge/service/query/{instance_id}"

headers = { "Accept": "text/event-stream" }
data = json.dumps({
    "text": " 我在哪里？",
    "knowledge_id": [ "test_knowledge" ],
    "knowledge_id_op": "and",
    "texts_id": [ "text_id_1", "text_id_2" ],
    "texts_id_op": "or",
    "scenes_name": [ " 场景一", " 场景二" ],
    "scenes_name_op": "orc",
    "extra_info": {
        "map": {
            "vsearch": " 敌人名",
            "prompt": " 当前玩家正位于“$name”",
            "name": " 位置名"
        }
    },
    "system_message": " 你是 AI 游戏助手，解决游戏里玩家遇到的各种问题。你回答的内容应该控制在 200 字以内。",
    "prompt_template": """< 指令 > 根据已知信息，简洁和专业的来回答问题。如果无法从中得到答案，请说 “根据已知信息无法
    < 已知信息 >{information}</已知信息 >
    < 问题 >{user_message}</问题 >"""
})
resp = requests.post(url, headers=headers, data=data, stream=True)

for chunk in resp.iter_content(chunk_size=1024):
    print(chunk.decode("utf-8"))

"""
# 完整的`message`:
# 您当前位于“位置名”。
# 根据提供的信息，从位置名出发，沿着道路走，
# 分岔路口往左走，看见山门进去就能找到敌人。

data: {
    "code": "ok",
    "data": {
        "text_ids": ["text_id_1", "text_id_2"]
    },
    "created_time": 1737944351.6639218,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
```

```
        "message": " 您"
    },
    "created_time": 1737944354.6639218,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 当前"
    },
    "created_time": 1737944354.7777965,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 位于"
    },
    "created_time": 1737944354.8891735,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " “位"
    },
    "created_time": 1737944355.09448,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 置"
    },
    "created_time": 1737944355.2059472,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 名"
    },
    "created_time": 1737944355.3139021,
    "fin": false
}


data: {
```

```
    "code": "ok",
    "data": {
        "message": ""。根据"
    },
    "created_time": 1737944355.5208077,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 提供的"
    },
    "created_time": 1737944355.6319518,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 信息"
    },
    "created_time": 1737944355.7348156,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ", 从"
    },
    "created_time": 1737944355.972084,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 位"
    },
    "created_time": 1737944356.088329,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 置"
    },
    "created_time": 1737944356.1884296,
    "fin": false
}
```

```
data: {
    "code": "ok",
    "data": {
        "message": " 名"
    },
    "created_time": 1737944356.41096,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 出发"
    },
    "created_time": 1737944356.5223246,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ", 沿着"
    },
    "created_time": 1737944356.7361913,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 道路"
    },
    "created_time": 1737944356.848111,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 走"
    },
    "created_time": 1737944356.9692416,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ", 分"
    },
    "created_time": 1737944357.1712604,
    "fin": false
}
```

```
data: {
    "code": "ok",
    "data": {
        "message": " 岔"
    },
    "created_time": 1737944357.291058,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 路口"
    },
    "created_time": 1737944357.3937087,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 往"
    },
    "created_time": 1737944357.5052245,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 左"
    },
    "created_time": 1737944357.616351,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 走"
    },
    "created_time": 1737944357.7116683,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ", 看见"
    },
    "created_time": 1737944357.9350371,
    "fin": false
```

```
}
data: {
    "code": "ok",
    "data": {
        "message": " 山"
    },
    "created_time": 1737944358.0611274,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 门"
    },
    "created_time": 1737944358.1713474,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 进去"
    },
    "created_time": 1737944358.269525,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 就能"
    },
    "created_time": 1737944358.3756146,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 找到"
    },
    "created_time": 1737944358.4946537,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 敌"
    },
    "created_time": 1737944358.5991426,
    "fin": false
```

```
}


data: {
    "code": "ok",
    "data": {
        "message": " 人"
    },
    "created_time": 1737944358.7039838,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": "。"
    },
    "created_time": 1737944358.9265351,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ""
    },
    "created_time": 1737944358.9265351,
    "fin": true
}


"""
```

7 ASR 服务

提供语音识别成文字的功能。

7.1 接口 `/asr/service/enable`

查询 ASR 服务是否启用。

7.1.1 Method `Get`

7.1.2 Content-Type `application/json`

7.1.3 参数

无

7.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
```

```
        "asr": bool 类型。是否启用 ASR 服务。
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

7.1.5 例子

```
# 查询 ASR 服务是否启用

import requests

url = f"http://127.0.0.1:9190/asr/service/enable"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"asr": True}}
```

7.2 接口 `/asr/service/enable/{enable}`

启用或停用 ASR 服务。

7.2.1 Method `Post`

7.2.2 Content-Type `application/json`

7.2.3 参数

1. `enable` ：Bool 类型。`true` 启用，`false` 停用。

7.2.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

7.2.5 例子

```
# 启用 ASR 服务

import requests

url = f"http://127.0.0.1:9190/asr/service/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

7.3 接口 `/asr/service/query/sample`

将音频采样点以数组的形式传给接口，返回识别到的内容。

7.3.1 Method `Post`

7.3.2 Content-Type `application/json`

7.3.3 参数

1. `sample_data` ：List 类型。音频采样点数组。
2. `hotwords` ：String 类型。热词、专有名词，多个热词以空格隔开。

7.3.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因", "data": {"output": " 识别到的内容"}}
```

返回的 `code` ：

1. `ok` ：成功。返回的内容在 `data` 中。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

7.3.5 例子

1. 通过加载音频文件，读取采样点数据发送给接口。

```
# 加载音频文件，读取采样点数据

import json
import requests
import librosa
import numpy as np

hotwords = " 特殊名字 专有名词 每个字都认识但放一起就看不懂的词"
mp3_file = "voice.mp3"

# sr 需要使用 16K，把音频文件重采样至 16K，因为模型配置的是 16K 采样率
ret = librosa.load(mp3_file, sr=16000)
waveform: np.ndarray = ret[0]
# waveform: array([0., 0., 0., ..., 0., 0., 0.], dtype=float32)

samplerate: int = ret[1]
# samplerate: 16000

sample_data: list = waveform.tolist()
# sample_data: [0.0, 0.0, 0.0, ..., 0.0, 0.0, 0.0]

url = "http://127.0.0.1:9190/asr/service/query/sample"
data = json.dumps({
    "sample_data": sample_data,
    "hotwords": hotwords
})
resp = requests.post(url, data=data)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"output": " 敌人的攻击频率不是很高。"}}
```

2. 录音获得采样点数据发送给接口。

```
# 录音获得采样点数据

import time
import json
import requests
import pyaudio
```

```
import numpy as np

hotwords = " 特殊名字 专有名词 每个字都认识但放一起就看不懂的词"

buf_size = 1024
p = pyaudio.PyAudio()
stream = p.open(format=pyaudio.paInt16,
                channels=1,
                rate=16000,
                frames_per_buffer=buf_size,
                input=True)

# 录音 5 秒钟
frames = []
record_seconds = 5
start_time = time.time()
while time.time() - start_time < record_seconds:
    data = stream.read(buf_size)
    frames.append(data)

stream.close()
p.terminate()

buffer: bytes = b"".join(frames)
sample_data: list = np.frombuffer(buffer, dtype=np.int16).tolist()
# sample_data: [0, 0, -1, ..., 0, 0, -1]

url = "http://127.0.0.1:9190/asr/service/query/sample"
data = json.dumps({
    "sample_data": sample_data,
    "hotwords": hotwords
})
resp = requests.post(url, data=data)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"output": " 敌人的攻击频率不是很高。"}}
```

7.4 接口 /asr/service/query/file

上传音频文件，返回识别到的内容。

7.4.1 Method Post

7.4.2 Content-Type multipart/form-data

7.4.3 参数

1. file ：Byte 类型。音频文件的 Byte 数据。
2. hotwords ：String 类型。热词、专有名词，多个热词以空格隔开。

7.4.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因", "data": {"output": " 识别到的内容"}}
```

返回的 code ：

1. ok ：成功。返回的内容在 data 中。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

7.4.5 例子

1. 把音频文件发送给接口。

```
# 加载音频文件

import requests

hotwords = " 特殊名字 专有名词 每个字都认识但放一起就看不懂的词"
mp3_file = "voice.mp3"

url = "http://127.0.0.1:9190/asr/service/query/file"
files = {"file": open(mp3_file, "rb")}
data = {"hotwords": hotwords}
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"output": " 敌人的攻击频率不是很高。"}}
```

2. 将录音保存为文件发送给接口。

```
# 录音保存为文件

import time
import requests
import wave
import pyaudio

hotwords = " 特殊名字 专有名词 每个字都认识但放一起就看不懂的词"

buf_size = 1024
p = pyaudio.PyAudio()
stream = p.open(format=pyaudio.paInt16,
                channels=1,
                rate=16000,
                frames_per_buffer=buf_size,
                input=True)

# 录音 5 秒钟
frames = []
record_seconds = 5
start_time = time.time()
while time.time() - start_time < record_seconds:
    data = stream.read(buf_size)
    frames.append(data)

stream.close()
p.terminate()

wav_file = "voice.wav"
f = wave.open(wav_file, "wb")
f.setnchannels(1)
f.setsampwidth(p.get_sample_size(pyaudio.paInt16))
f.setframerate(16000)
f.writeframes(b"".join(frames))
f.close()

url = "http://127.0.0.1:9190/asr/service/query/file"
files = {"file": open(wav_file, "rb")}
data = {"hotwords": hotwords}
resp = requests.post(url, data=data, files=files)
```

```
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"output": " 敌人的攻击频率不是很高。"}}
```

8 VLM 服务

提供视觉语言模型的功能。

8.1 接口 `/vlm/service/enable`

查询 VLM 服务是否启用。

8.1.1 Method `Get`

8.1.2 Content-Type `application/json`

8.1.3 参数

无

8.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "vlm": bool 类型。是否启用 VLM 服务。
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

8.1.5 例子

```
# 查询 VLM 服务是否启用

import requests

url = f"http://127.0.0.1:9190/vlm/service/enable"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"vlm": True}}
```

8.2 接口 `/vlm/service/enable/{enable}`

启用或停用 VLM 服务。

8.2.1 Method `Post`

8.2.2 Content-Type `application/json`

8.2.3 参数

1. `enable` ：Bool 类型。`true` 启用，`false` 停用。

8.2.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

8.2.5 例子

```
# 启用 VLM 服务

import requests

url = f"http://127.0.0.1:9190/vlm/service/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

8.3 接口 `/vlm/service/query`

将文本、图片传给接口，返回视觉语言模型的回答内容。

8.3.1 Method `Post`

8.3.2 Content-Type `multipart/form-data`

8.3.3 参数

1. `text` ：String 类型。`必须` 。以自然语言的形式提出的问题。
2. `images` ：List[Byte] 类型。`可选` 。提供给模型识别或理解的图片。

8.3.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "message": " 增量回答的内容"
    },
    "created_time": float 类型，回答创建时间（秒），
    "fin": bool 类型，流式返回是否结束
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

8.3.5 例子

```
# 上传图片文件，询问图片内容

import requests

pic = r"C:\Users\Public\Pictures\场景 1_ 图片 1.jpg"
```

```
url = f"http://127.0.0.1:9190/vlm/service/query"

headers = { "Accept": "text/event-stream" }
files = [ ("images", open(pic, "rb")) ]
data = { "text": " 描述一下这张图片。" }

resp = requests.post(url, headers=headers, data=data, files=files, stream=True)

for chunk in resp.iter_content(chunk_size=1024):
    print(chunk.decode("utf-8"))

"""
# 完整的`message`:
# 这是一张黑色的图片。

data: {
    "code": "ok",
    "data": {
        "message": " 这是"
    },
    "created_time": 1754379370.1486354,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 一张"
    },
    "created_time": 1754379370.4633887,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 黑色"
    },
    "created_time": 1754379370.561034,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 的"
    },
    "created_time": 1754379370.6291597,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
```

```
        "message": " 图片"
    },
    "created_time": 1754379370.6913445,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": "。"
    },
    "created_time": 1754379370.7520256,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ""
    },
    "created_time": 1754379370.753614,
    "fin": true
}

"""
```

9 OCR 服务

提供图片识别文字的功能。

9.1 接口 `/ocr/service/enable`

查询 OCR 服务是否启用。

9.1.1 Method `Get`

9.1.2 Content-Type `application/json`

9.1.3 参数

无

9.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "ocr": bool 类型。是否启用 OCR 服务。
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

9.1.5 例子

```
# 查询 OCR 服务是否启用

import requests

url = f"http://127.0.0.1:9190/ocr/service/enable"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"ocr": True}}
```

9.2 接口 /ocr/service/enable/{enable}

启用或停用 OCR 服务。

9.2.1 Method Post

9.2.2 Content-Type application/json

9.2.3 参数

1. enable ：Bool 类型。true 启用，false 停用。

9.2.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

9.2.5 例子

```
# 启用 OCR 服务

import requests

url = f"http://127.0.0.1:9190/ocr/service/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

9.3 接口 /ocr/service/query/file

将图片传给接口，返回识别到的内容。

9.3.1 Method Post

9.3.2 Content-Type multipart/form-data

9.3.3 参数

1. file ：Byte 类型。图片文件的 Byte 数据。
2. return_boxes ：Bool 类型。是否返回文本对应的矩形框。矩形框的四个点坐标顺序是左上角、右上角、右下角、左下角，原点为图片左上角。

9.3.4 返回

格式：

```
{
  "code": " 返回代码",
  "msg": " 失败原因",
  "data": {
    "output": [
        [文本, 分数]
    ],
    "boxes": [
        [
            [左上角 X, Y]
            [右上角 X, Y]
            [右下角 X, Y]
            [左下角 X, Y]
        ]
    ]
  }
}
```

返回的 `code` ：

1. `ok` ：成功。返回的内容在 `data` 中。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

9.3.5 例子

```
# 上传图片文件，返回识别到的文本内容

import requests

pic = r"C:\Users\Public\Pictures\场景 1_ 图片 1.jpg"

url = f"http://127.0.0.1:9190/ocr/service/query/file"

files = { "file": open(pic, "rb") }
data = { "return_boxes": True }

resp = requests.post(url, data=data, files=files)
resp_data = resp.json()

""" resp_data:
{
    "code": "ok",
    "data": {
        "output": [
            [ " 漂浮", 0.9986114501953125 ],
            [ " 轻盈泡泡", 0.9883835315704346 ],
            [ " 能够延长跳跃后的滞空时间。", 0.9839507341384888 ]
        ],
        "boxes": [
            [
                [ 138, 901 ],
                [ 208, 901 ],
                [ 208, 954 ],
                [ 138, 954 ]
            ],
            [
```

```
                [ 169, 959 ],
                [ 570, 962 ],
                [ 569, 1076 ],
                [ 169, 1073 ]
            ],
            [
                [ 93, 1145 ],
                [ 436, 1145 ],
                [ 436, 1177 ],
                [ 93, 1177 ]
            ]
        ]
    }
}
"""
```

10 LLM 服务

提供语言模型的功能。

10.1 接口 `/llm/service/enable`

查询 LLM 服务是否启用。

10.1.1 Method `Get`

10.1.2 Content-Type `application/json`

10.1.3 参数

无

10.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "llm": bool 类型。是否启用 LLM 服务。
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

10.1.5 例子

```
# 查询 LLM 服务是否启用

import requests

url = f"http://127.0.0.1:9190/llm/service/enable"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"llm": True}}
```

10.2 接口 /llm/service/enable/{enable}

启用或停用 LLM 服务。
该接口和 知识库 的 /knowledge/service/llm/enable/{enable} 接口共享。

10.2.1 Method Post

10.2.2 Content-Type application/json

10.2.3 参数

1. enable ：Bool 类型。true 启用，false 停用。

10.2.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

10.2.5 例子

```
# 启用 LLM 服务

import requests

url = f"http://127.0.0.1:9190/llm/service/enable/true"
resp = requests.post(url)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

10.3 接口 /llm/service/query

将文本传给接口，返回语言模型的回答内容。
相比 知识库 的 /knowledge/service/query/{instance_id} 接口具有更灵活的使用方式。

10.3.1 Method Post

10.3.2 Content-Type application/json

10.3.3 参数

1. prompt ：String 类型。可选 。以自然语言的形式提出的问题。可以不传，此时模型会接着 histories 最后一条对话记录继续回答。
2. system_message ：String 类型。可选 。如果模型不支持 system_message ，会被忽略。
3. histories ：List[Dict[String, String]] 类型。可选 。提供给模型的对话历史。如果提供了 prompt ，在下次对话时，需要把 prompt 追加到对话历史中。
4. tools ：String 类型。可选 。提供给模型可供调用的工具列表。如果模型不支持 tools ，会被忽略。
5. params ：Dict[String, Any] 类型。可选 。提供给模型的参数。如果配置文件中有相同的参数，以此传入的参数为准。

10.3.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "message": " 增量回答的内容"
    },
    "created_time": float 类型，回答创建时间（秒），
    "fin": bool 类型，流式返回是否结束
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

10.3.5 例子

```
# 调用工具的问答例子

import json
import requests

# 获取游戏攻略的工具函数
def get_walk_through(game: str, count: int | str = 5):
    return f" 这是游戏《{game}》的{count}篇攻略：沿着路走，打败所有敌人，即可通关。"

# 传给模型的工具定义
tools = [
    {
        "name": "get_walk_through",
        "description": " 获取游戏相关攻略",
        "input_schema": {
            "properties": {
                "game": {
                    "description": " 游戏名",
                    "title": "Game",
                    "type": "string"
                },
                "count": {
                    "anyOf": [
                        {
                            "type": "string"
                        },
                        {
                            "type": "integer"
                        }
                    ],
                    "description": " 返回攻略的个数",
                    "title": "Count"
                }
            },
            "required": [
                "game"
            ],
            "type": "object"
        }
    }
]
```

```
# 模型参数
llm_params = {
    "top_k": 1,
    "top_p": 1.0,
    "temperature": 0.7,
    "repetition_penalty": 1.0,
    "max_new_tokens": 256,
    "do_sample": True
}

system_message = " 你是 AI 游戏助手。"
histories = [
    { "role": "user", "content": " 你是谁？" },
    { "role": "assistant", "content":
    " 我是 AI 游戏助手，可以为您提供游戏相关的帮助和信息。请问您需要什么帮助？" },
    { "role": "user", "content": " 帮我查一下 Game 的攻略，给我一个就够了。"},
    { "role": "assistant", "content":
    "{\"name\": \"get_walk_through\", \"arguments\": {\"game\": \"Game\", \"count\": \"1\"}}"},
    { "role": "tool", "name": "get_walk_through", "content":
    " 这是游戏《Game》的 1 篇攻略：沿着路走，打败所有敌人，即可通关。" }
]

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/llm/service/query"

headers = { "Accept": "text/event-stream" }
data = json.dumps({
    "prompt": None,
    "system_message": system_message,
    "histories": histories,
    "tools": tools,
    "params": llm_params
})
resp = requests.post(url, headers=headers, data=data, stream=True)

for chunk in resp.iter_content(chunk_size=1024):
    print(chunk.decode("utf-8"))

"""
# 完整的`message`:
# 沿着路走，打败所有敌人，即可通关。

data: {
    "code": "ok",
    "data": {
        "message": " 沿着"
    },
    "created_time": 1754469796.889811,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 路"
    },
    "created_time": 1754469796.951177,
```

```
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 走"
    },
    "created_time": 1754469797.0109732,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ", 打"
    },
    "created_time": 1754469797.1292036,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 败"
    },
    "created_time": 1754469797.1891208,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 所有"
    },
    "created_time": 1754469797.2492073,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 敌"
    },
    "created_time": 1754469797.3099892,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 人"
```

```
    },
    "created_time": 1754469797.370155,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ", 即可"
    },
    "created_time": 1754469797.4887445,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": " 通关"
    },
    "created_time": 1754469797.5467422,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": "。"
    },
    "created_time": 1754469797.6672153,
    "fin": false
}


data: {
    "code": "ok",
    "data": {
        "message": ""
    },
    "created_time": 1754469797.6682518,
    "fin": true
}

"""
```

11 OpenAI 兼容接口

本服务提供与 OpenAI API 兼容的接口，支持直接使用 OpenAI 官方 SDK 或任何兼容 OpenAI 协议的客户端进行调用。

基础 URL: `http://<host>:<port>/v1`

11.1 接口 /v1/chat/completions

11.1.1 Method Post

11.1.2 Content-Type application/json

11.1.3 参数

参数	类型	必需	默认值	说明
model	string	否	"default"	模型名称，仅用于兼容性，不影响路由
messages	array	是	-	消息列表，支持文本和图片内容
temperature	float	否	-	采样温度
top_p	float	否	-	核采样参数
max_tokens	int	否	-	最大生成 token 数，映射为内部 max_new_tokens
max_completion_tokens	int	否	-	最大生成 token 数（优先于 max_tokens）
stream	bool	否	false	是否流式返回
tools	array	否	-	工具定义列表（function calling）
stop	string/array	否	-	停止词（暂不支持，仅接受）
n	int	否	1	生成数量（仅支持 n=1）
repetition_penalty	float	否	-	重复惩罚参数（扩展参数）
enable_thinking	bool	否	-	是否开启思考模式，仅对 VLM 多模态请求生效（扩展参数）

11.1.4 消息格式

每条消息包含以下字段：

字段	类型	必需	说明
role	string	是	角色：system 、user 、assistant 、tool
content	string/array	否	消息内容，支持纯文本或多模态数组
name	string	否	发送者名称
tool_calls	array	否	工具调用（assistant 消息）
tool_call_id	string	否	工具调用 ID（tool 消息）

多模态消息（图片）格式：

```
{
  "role": "user",
  "content": [
    {"type": "text", "text": " 请描述这张图片"},
    {"type": "image_url", "image_url": {"url": "..."}}
  ]
}
```

当消息中包含图片时，自动路由到 VLM 服务处理；纯文本消息由 LLM 服务处理。

VLM 多模态请求会将 `temperature` 、 `top_p` 、 `max_tokens` / `max_completion_tokens` 、 `repetition_penalty` 、 `enable_thinking` 等生成参数透传给 VLM 服务。 `enable_thinking` 默认关闭思考模式（即直接作答），可通过请求参数 `"enable_thinking": true` 或在 `vlm.json` 的 `args` 中配置开启；开启后模型先输出思考过程再给出回答。

11.1.5 返回（非流式）

```
{
  "id": "chatcmpl-...",
  "object": "chat.completion",
  "created": 1712345678,
  "model": "default",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "content": " 回复内容"
    },
    "logprobs": null,
    "finish_reason": "stop"
  }],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 5,
    "total_tokens": 15
  }
}
```

11.1.6 返回（流式）

流式返回 SSE 格式，每个 chunk 格式如下：

```
data: {"id":"chatcmpl-...","object":"chat.completion.chunk","created":1712345678,"model":"default","choices":

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","created":1712345678,"model":"default","choices":

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","created":1712345678,"model":"default","choices":

data: [DONE]
```

11.1.7 错误返回

```
{
  "error": {
    "message": "LLM service is disabled.",
    "type": "server_error",
    "param": null,
    "code": null
  }
}
```

11.1.8 例子

纯文本对话

```
curl -X POST http://localhost:9190/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "default",
```

```
    "messages": [{"role": "user", "content": " 你好"}]
  }'
```

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:9190/v1", api_key="not-needed")
response = client.chat.completions.create(
    model="default",
    messages=[{"role": "user", "content": " 你好"}]
)
print(response.choices[0].message.content)
```

流式对话

```
curl -N -X POST http://localhost:9190/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "default",
    "messages": [{"role": "user", "content": " 讲个故事"}],
    "stream": true
  }'
```

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:9190/v1", api_key="not-needed")
stream = client.chat.completions.create(
    model="default",
    messages=[{"role": "user", "content": " 讲个故事"}],
    stream=True
)
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="")
```

带系统提示和历史记录

```
curl -X POST http://localhost:9190/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "default",
    "messages": [
      {"role": "system", "content": " 你是一个游戏助手，用中文回答。"},
      {"role": "user", "content": "BOSS 在哪里？"},
      {"role": "assistant", "content": "BOSS 在山门后面。"},
      {"role": "user", "content": " 怎么去山门？"}
    ]
  }'
```

带图片的多模态对话（VLM）

```
curl -X POST http://localhost:9190/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "default",
    "messages": [{
      "role": "user",
      "content": [
        {"type": "text", "text": " 图片里是什么？"},
        {"type": "image_url", "image_url": {"url": "...="}}
      ]
    }]
  }'
```

开启 VLM 思考模式

```
curl -X POST http://localhost:9190/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "default",
    "messages": [{
      "role": "user",
      "content": [
        {"type": "text", "text": " 图片里是什么？"},
        {"type": "image_url", "image_url": {"url": "...="}}
      ]
    }],
    "enable_thinking": true
  }'
```

带工具调用（function calling）

```
curl -X POST http://localhost:9190/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "default",
    "messages": [{"role": "user", "content": " 查看天气"}],
    "tools": [{
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": " 获取天气",
        "parameters": {"type": "object", "properties": {"city": {"type": "string"}}}
      }
    }]
  }'
```

11.2 接口 /v1/embeddings

11.2.1 Method Post

11.2.2 Content-Type application/json

11.2.3 参数

参数	类型	必需	默认值	说明
input	string/array	是	-	需要向量化的文本，支持单个字符串或字符串数组
model	string	否	"default"	模型名称
encoding_format	string	否	"float"	编码格式

11.2.4 返回

```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [0.01, 0.02, ...]
    }
  ],
  "model": "default",
```

```
  "usage": {
    "prompt_tokens": 2,
    "total_tokens": 2
  }
}
```

11.2.5 例子

单个文本

```
curl -X POST http://localhost:9190/v1/embeddings \
  -H "Content-Type: application/json" \
  -d '{"input": "hello world", "model": "default"}'
```

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:9190/v1", api_key="not-needed")
response = client.embeddings.create(
    model="default",
    input="hello world"
)
print(len(response.data[0].embedding))
```

批量文本

```
curl -X POST http://localhost:9190/v1/embeddings \
  -H "Content-Type: application/json" \
  -d '{"input": ["hello", "world", "test"], "model": "default"}'
```

11.3 接口 /v1/audio/transcriptions

11.3.1 Method Post

11.3.2 Content-Type multipart/form-data

11.3.3 参数

参数	类型	必需	默认值	说明
file	file	是	-	音频文件，支持 WAV 等格式
model	string	否	"default"	模型名称
language	string	否	-	音频语言代码
prompt	string	否	-	可选提示文本
temperature	float	否	-	采样温度
response_format	string	否	"json"	返回格式：json 或 text

11.3.4 返回

```
{"text": " 转写的文本内容"}
```

11.3.5 例子

```
curl -X POST http://localhost:9190/v1/audio/transcriptions \
  -F "file=@audio.wav" \
  -F "model=default"
```

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:9190/v1", api_key="not-needed")
```

```python
with open("audio.wav", "rb") as f:
    response = client.audio.transcriptions.create(
        model="default",
        file=f
    )
print(response.text)
```

11.4 接口 /v1/models

11.4.1 Method Get

11.4.2 Content-Type application/json

11.4.3 参数

无。

11.4.4 返回

```json
{
  "object": "list",
  "data": [
    {
      "id": "default",
      "object": "model",
      "created": 1712345678,
      "owned_by": "game-assistant"
    }
  ]
}
```

11.4.5 例子

```bash
curl http://localhost:9190/v1/models
```

```python
from openai import OpenAI
client = OpenAI(base_url="http://localhost:9190/v1", api_key="not-needed")
models = client.models.list()
print(models.data[0].id)
```

11.5 流式返回说明

/v1/chat/completions 接口在 stream=true 时返回标准 SSE 格式：

1. 每个 chunk 以 data: 开头，后跟 JSON 格式的 chat.completion.chunk 对象
2. 第一个 chunk 的 delta 中包含 role: "assistant"
3. 后续 chunk 的 delta 中只包含 content 增量
4. 当 finish_reason 为 "stop" 时表示生成结束
5. 流结束时返回 data: [DONE]

11.6 限制说明

1. n > 1 不被支持，会返回 400 错误
2. stop 、seed 、logprobs 、presence_penalty 、frequency_penalty 参数目前被接受但不生效
3. usage 中的 token 数为估算值，非精确计算
4. 多模态对话需要 VLM 服务已启用
5. 语音转写需要 ASR 服务已启用